Lab 38 — Identity & Access Management with PAM and sudo

> **SY0-701 Objective 4.6** — Given a scenario, implement and maintain identity and access management.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~20 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: provision role-based users, give each role only the privileges it needs, enforce a password and lockout policy, and cleanly de-provision a leaver.

You'll build: three RBAC roles with scoped sudo, a DAC/ACL-protected share, a just-in-time privilege grant that revokes itself, a pwquality + password-history standard, a pam_faillock lockout, a pam_time time-of-day rule and an offboarded account.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab required; tools installed in step 1.

Background

- The identity lifecycle runs from **provisioning** (create the account from an approved request, after identity proofing) through **permission assignment**, periodic review, to **de-provisioning** when the person leaves. Accounts left behind are orphaned accounts and a favourite attacker foothold.
- **DAC** (discretionary) lets the owner set permissions, as Linux mode bits and ACLs do. **RBAC** grants rights to roles (Linux groups) rather than to people. **Rule-based** access applies system-wide rules, such as a time-of-day restriction. **MAC** uses labels users cannot override.
- **Least privilege** means each account gets only the commands it needs. sudoers can scope privilege down to a single command line, and **just-in-time (JIT)** permissions grant elevated rights only for a short, approved window.
- **Password concepts** on the exam: length beats complexity, block reuse with history, and understand expiration (maximum age) versus minimum age. NIST SP 800-63B now discourages forced periodic changes unless a compromise is suspected.
- **PAM** (Pluggable Authentication Modules) stacks modules in four phases: auth, account, password and session. pam_pwquality, pam_pwhistory, pam_faillock and pam_time are all PAM modules.

Steps

1. Install the tools

Install ACL support, the pwquality module and its `pwscore` tester, and `pamtester`, which lets you exercise a PAM stack from the command line without logging in.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y sudo acl libpam-pwquality libpwquality-tools pamtester
grep pwquality /etc/pam.d/common-password
```

```
password    requisite           pam_pwquality.so retry=3
```

2. Provision role groups and users

Each account is created from a request ticket (recorded in the comment field), placed in exactly one role group and given an initial credential. The loop is idempotent, so re-running it does not fail.

```
for g in webops dbops auditor; do getent group "$g" >/dev/null || groupadd "$g"; done
while IFS=: read -r u g gecos; do
  id "$u" >/dev/null 2>&1 || useradd -m -s /bin/bash -G "$g" -c "$gecos" "$u"
  echo "$u:Labs-Pass-2026" | chpasswd
done <<'EOF'
alice:webops:Alice Tan - Web Ops - REQ-1001
bob:dbops:Bob Lim - DB Ops - REQ-1002
carol:auditor:Carol Ng - Audit - REQ-1003
EOF
for u in alice bob carol; do id "$u"; getent passwd "$u" | cut -d: -f5; done
```

3. Assign file permissions with DAC and ACLs

The audit share belongs to the auditor role (DAC via owner/group bits). Bob gets a one-off ACL exception, which works but is exactly how privilege creep starts, so it must be tracked and reviewed.

```
mkdir -p /srv/reports && echo "Q3 audit findings (synthetic data)" > /srv/reports/q3.txt
chown -R root:auditor /srv/reports && chmod 750 /srv/reports && chmod 640
/srv/reports/q3.txt
setfacl -m u:bob:rx /srv/reports && setfacl -m u:bob:r /srv/reports/q3.txt
getfacl -p /srv/reports/q3.txt
su - carol -c 'cat /srv/reports/q3.txt'
su - bob   -c 'cat /srv/reports/q3.txt'
su - alice -c 'cat /srv/reports/q3.txt' || echo "alice DENIED: no role membership and no
ACL"
```

4. Scope sudo per role (RBAC and least privilege)

Each role receives only the commands its job requires. Always validate sudoers with `visudo -c`, because a syntax error can break privilege escalation for everyone.

```
cat > /etc/sudoers.d/10-roles <<'EOF'
# RBAC: each role gets only the commands its job needs (least privilege)
%webops  ALL=(root) NOPASSWD: /usr/bin/systemctl status cron, /usr/bin/systemctl restart
cron
%dbops   ALL=(root) NOPASSWD: /usr/bin/du -sh /var/lib/*
%auditor ALL=(root) NOPASSWD: /usr/bin/last
EOF
chmod 440 /etc/sudoers.d/10-roles && visudo -cf /etc/sudoers.d/10-roles
sudo -l -U alice | tail -3
su - alice -c 'sudo -n systemctl status cron --no-pager | head -3'
su - alice -c 'sudo -n cat /etc/shadow' || echo "alice DENIED: command not in her role"
```

```
/etc/sudoers.d/10-roles: parsed OK
sudo: a password is required
```

```
alice DENIED: command not in her role
```

5. Grant a just-in-time privilege that revokes itself

For change CHG-2042, Carol needs to restart cron once. Instead of a standing privilege, grant it for two minutes and schedule its automatic removal. This is the JIT pattern that PAM (privileged access management) tools automate.

```
cat > /etc/sudoers.d/90-jit-carol <<'EOF'
# JIT grant for CHG-2042 - expires automatically after 2 minutes
carol ALL=(root) NOPASSWD: /usr/bin/systemctl restart cron
EOF
chmod 440 /etc/sudoers.d/90-jit-carol && visudo -cf /etc/sudoers.d/90-jit-carol
systemd-run --unit=jit-revoke-carol --on-active=120 /bin/rm -f /etc/sudoers.d/90-jit-carol
su - carol -c 'sudo -n systemctl restart cron && echo "JIT privilege used by carol"'
systemctl list-timers --all --no-pager | grep jit-revoke
```

6. Enforce a password policy (length, complexity, reuse, age)

pwquality enforces length and character classes when a password changes, pam_pwhistory blocks reuse of the last five passwords, and `chage` sets maximum age (expiration), minimum age and the warning period.

```
cp -n /etc/security/pwquality.conf /root/pwquality.conf.bak
cat >> /etc/security/pwquality.conf <<'EOF'
# C628 lab password standard
minlen = 14
minclass = 3
maxrepeat = 3
dictcheck = 1
usercheck = 1
EOF
grep -q pam_pwhistory /etc/pam.d/common-password || \
  sed -i '/^password.*pam_unix.so/i password required pam_pwhistory.so remember=5
use_authtok' /etc/pam.d/common-password
grep -E 'pwquality|pwhistory|pam_unix' /etc/pam.d/common-password
for p in 'Summer2026' 'aaaaaaaaaaaaaaa1' 'correct-Horse-battery-staple-9'; do echo "==
$p"; echo "$p" | pwscore 2>&1; done
chage -M 365 -m 1 -W 14 alice && chage -l alice
```

```
== Summer2026
Password quality check failed:
 The password is shorter than 14 characters
== aaaaaaaaaaaaaaa1
Password quality check failed:
 The password contains less than 3 character classes
== correct-Horse-battery-staple-9
100
```

7. Lock out repeated failures with pam_faillock

Account lockout defeats online brute force. Configure five failures within 15 minutes to lock for 10 minutes, wire pam_faillock around pam_unix, and test it through a dedicated PAM service so the system login stack stays untouched.

```
cat > /etc/security/faillock.conf <<'EOF'
deny = 5
fail_interval = 900
unlock_time = 600
EOF
cp -n /etc/pam.d/common-auth /root/common-auth.bak
grep -q pam_faillock /etc/pam.d/common-auth || sed -i \
  -e '/^auth.*pam_unix.so/i auth    requisite                       pam_faillock.so
preauth' \
  -e '/^auth.*pam_unix.so/a auth    [default=die]                   pam_faillock.so
authfail' \
  -e '/^auth.*pam_unix.so/a auth    sufficient                      pam_faillock.so
authsucc' \
  /etc/pam.d/common-auth
grep -q pam_faillock /etc/pam.d/common-account || echo 'account required pam_faillock.so'
>> /etc/pam.d/common-account
printf '@include common-auth\n@include common-account\n' > /etc/pam.d/labauth
grep -E 'faillock|pam_unix' /etc/pam.d/common-auth
for i in 1 2 3 4 5 6; do echo 'wrong-guess' | pamtester labauth alice authenticate; done
faillock --user alice
echo 'Labs-Pass-2026' | pamtester labauth alice authenticate || echo "alice LOCKED OUT
even with the right password"
faillock --user alice --reset
echo 'Labs-Pass-2026' | pamtester labauth alice authenticate
```

```
pamtester: Authentication failure
...
The account is locked due to 5 failed logins.
alice LOCKED OUT even with the right password
pamtester: successfully authenticated
```

8. Restrict logins by time of day with pam_time

A rule-based control: Alice (web ops) may authenticate on weekdays 08:00 to 19:00 only, and Carol is blocked for this service at all times. In `time.conf`, `Wk` means Monday to Friday, `Wd` means the weekend and `Al` means every day.

```
cat >> /etc/security/time.conf <<'EOF'
# C628 lab rules: services;ttys;users;times
labauth;*;alice;Wk0800-1900
labauth;*;carol;!Al0000-2400
EOF
grep -q pam_time /etc/pam.d/labauth || echo 'account required pam_time.so' >>
/etc/pam.d/labauth
date
pamtester labauth alice acct_mgmt || echo "alice is outside her permitted hours right now"
pamtester labauth carol acct_mgmt || echo "carol DENIED by the time-of-day rule"
```

9. De-provision a leaver

Bob has resigned. Offboarding must disable the credential, expire the account, remove the login shell, strip group and ACL entitlements, end running sessions and preserve the home directory for the records retention period.

```
usermod -L -e 1 -s /usr/sbin/nologin bob
gpasswd -d bob dbops
setfacl -x u:bob /srv/reports/q3.txt && setfacl -x u:bob /srv/reports
pkill -KILL -u bob || true
tar -czf "/root/offboard-bob-$(date +%F).tgz" -C /home bob
passwd -S bob; chage -l bob | grep 'Account expires'; id bob
echo 'Labs-Pass-2026' | pamtester labauth bob authenticate || echo "bob can no longer
authenticate"
```

```
bob L 10/28/2026 0 99999 7 -1
Account expires                     : Jan 02, 1970
uid=1002(bob) gid=1002(bob) groups=1002(bob)
bob can no longer authenticate
```

Test it

Wait two minutes after step 5, then confirm the JIT grant has gone, the roles still work and the leaver is fully disabled.

```
ls /etc/sudoers.d/
su - carol -c 'sudo -n systemctl restart cron' || echo "PASS: JIT privilege expired"
su - alice -c 'sudo -n systemctl status cron --no-pager | head -1'
passwd -S bob | awk '{print "bob password status:", $2}'
getfacl -p /srv/reports/q3.txt | grep -c 'user:bob' || echo "PASS: no ACL left for bob"
```

Expected: `90-jit-carol` is no longer listed, Carol's restart is refused, Alice can still check cron, Bob shows status `L` (locked) and has no ACL entry.

Troubleshooting

Symptom	Fix
Every pamtester authentication fails after step 7, even with the right password	The jump counts in common-auth are wrong on your image. Restore with `cp /root/common-auth.bak /etc/pam.d/common-auth` and check that `pam_unix` has `success=1` and is followed by the `authfail` then `authsucc` lines.
`visudo` reports a syntax error	Fix the file before closing the shell; a broken sudoers file can stop all sudo use. Commands in sudoers must be full paths such as `/usr/bin/systemctl`.
`systemd-run` fails with "System has not been booted with systemd"	Use a full VM playground, or revoke manually with `rm -f /etc/sudoers.d/90-jit-carol` after two minutes.

Challenge

Write an `access-review.sh` script that lists every user with a UID of 1000 or more, their groups, any sudoers entries that name them or their groups, their last password change (`chage -l`) and any ACL entries under `/srv`. This is the evidence a manager needs to attest (recertify) that each entitlement is still required.

Reflection

A temporary ACL for Bob and a permanent JIT grant both "worked". Which access control principle does each one violate if nobody reviews it, and which automated control from this lab prevents that drift?

Exam tips

- RBAC grants permissions to roles; rule-based applies the same rule to everyone (time of day, firewall ACL). Do not confuse the two acronyms.
- DAC = owner decides (file permissions); MAC = labels and clearances the user cannot change (SELinux, government classification).
- Minimum password age stops a user cycling through the history list to reuse an old password; maximum age is expiration.
- Account lockout (pam_faillock) defends against brute force but not against password spraying, which stays under the threshold per account.

References

- NIST SP 800-63B Digital Identity Guidelines (passwords) — https://pages.nist.gov/800-63-3/sp800-63b.html
- NIST SP 800-162 Guide to ABAC — https://csrc.nist.gov/publications/detail/sp/800-162/final
- Linux-PAM documentation — https://github.com/linux-pam/linux-pam
- pam_faillock(8), pam_pwquality(8), pam_time(8) and sudoers(5) man pages — https://manpages.ubuntu.com
- LabEx CompTIA Security+ training labs (user identities, password policies) — https://github.com/labex-labs/comptia-security-plus-training-labs

---